Proyecto Final Integrador – Análisis de Algoritmos (SIST0175)

Docente Juan David Rodríguez Martínez

1. Datos generales

Cada equipo de 3 estudiantes diseña, implementa, analiza y valida experimentalmente un sistema algorítmico de escala reducida sobre un caso de ingeniería propio, integrando los temas del curso en seis componentes obligatorios.

Elemento	Detalle
Asignatura	Análisis de Algoritmos (SIST0175), VI semestre
Programa	Ciencias de la Computación e Inteligencia Artificial
Equipos	8 grupos de 3 estudiantes, un caso por grupo
Duración	3 sesiones de laboratorio (2 h c/u) + sesión 4 de entrega y sustentación (2 h), complementadas con trabajo autónomo (5 h semanales por estudiante según el CPA)
Lenguaje	Python 3.10+ (NumPy permitido). C++ opcional para módulos de rendimiento
Peso en el curso	El proyecto se compone de 2 entregas: la primera entrega se promediará con los demás trabajos del corte, los cuales valen 20% de este, la entrega final constituye el examen final del Corte 3 (20 % de la nota final), la cual se compone de sustentación verbal (10%) y envío de evidencias (10%).

Propósito. El objetivo no es construir un sistema de producción, sino demostrar que el equipo sabe elegir una estrategia algorítmica, probar su corrección y su complejidad, y verificar con experimentos que la teoría predice el comportamiento real. Los dominios (búsqueda legal, logística, genómica, etc.) son el contexto que motiva los algoritmos; los datos son sintéticos y de tamaño manejable en un computador portátil.

2. Competencias y resultados de aprendizaje

El proyecto evidencia las tres unidades de competencia del contenido programático (CPA) con los alcances formativos allí definidos.

U.C.	Alcance formativo (CPA)	Cómo se evidencia en el proyecto
2.2	Comprender las clases de complejidad y las técnicas de análisis de algoritmos	Cotas O, Ω y Θ de cada componente; mejor, peor y caso esperado; recurrencias resueltas con el Teorema Maestro; análisis amortizado por el método contable

U.C.	Alcance formativo (CPA)	Cómo se evidencia en el proyecto
3.3	Analizar la complejidad para optimizar tiempos y uso de memoria, y comunicar resultados	Experimentos que contrastan la predicción teórica con tiempos, memoria y número de E/S medidos; informe y sustentación
4.3	Desarrollar soluciones con diferentes estrategias algorítmicas	Implementación de divide y vencerás, programación dinámica o voraz con prueba de optimalidad, aleatorizado o backtracking, estructuras en memoria secundaria y un módulo de aproximación, paralelismo o IA

Al terminar, cada estudiante debe poder:

1. Explicar el mejor caso, el peor caso y el caso esperado de los algoritmos de su sistema y qué datos los provocan.
2. Resolver la recurrencia de su algoritmo recursivo e identificar el caso del Teorema Maestro que aplica.
3. Justificar por qué su regla voraz es óptima, o por qué un problema tiene subestructura óptima y subproblemas superpuestos.
4. Predecir el número de transferencias de bloque de su estructura en memoria secundaria y comprobarlo midiendo.
5. Diseñar un experimento que valide una hipótesis de complejidad y leer correctamente una gráfica log-log.

3. Estructura común: seis componentes obligatorios

Todos los grupos implementan exactamente seis componentes (C1 a C6) con el mismo tipo de exigencia; solo cambia el algoritmo concreto según el caso. Se sugiere repartir dos componentes por estudiante, pero cada integrante debe poder explicar los seis.

Comp.	Pilar	Qué se exige	Evidencia mínima
C1	I. Fundamentos	Un algoritmo de divide y vencerás del caso	Recurrencia T(n) planteada desde el código, resuelta con el Teorema Maestro (indicar caso, a, b, f(n)) y validada empíricamente
C2	I. Fundamentos	Una estructura dinámica con redimensionamiento	Análisis amortizado por el método contable (créditos por operación) y medición del costo acumulado
C3	II. Diseño	Estrategia óptima: programación dinámica o algoritmo voraz	DP: recurrencia, subestructura óptima, subproblemas superpuestos, tabla. Voraz: prueba de optimalidad por intercambio o contraejemplo cuando no es óptimo
C4	II. Diseño	Estrategia complementaria:	Aleatorizado: análisis del caso esperado o de la probabilidad de error. Backtracking: árbol de

Comp.	Pilar	Qué se exige	Evidencia mínima
		aleatorizado o backtracking	búsqueda, criterio de poda y comparación con la versión sin poda
C5	III. Memoria secundaria	Una estructura o algoritmo en disco sobre el disco simulado común	Fórmula de costo en E/S según el modelo de memoria externa (parámetros N, M, B) y conteo real de lecturas y escrituras de bloque
C6	IV. Complejidad, IA y paralelismo	Algoritmo aproximado, paralelo o de IA con garantía formal	Enunciado y prueba (o argumento formal guiado) de la garantía: razón de aproximación, trabajo y profundidad, costo de comunicación o cota de muestras; verificación experimental

Bonificación opcional (hasta +0,3 sobre 5,0). Cada caso propone una estructura comprimida o sucinta adicional (FM-Index, árbol wavelet, Elias-Fano, árbol R, etc.). Solo se califica si C1 a C6 están completos.

4. Reglas técnicas comunes

Todos los grupos usan el mismo disco simulado y el mismo protocolo de medición, para que los resultados sean comparables y la calificación sea equitativa.

4.1 Disco simulado. El componente C5 se implementa sobre una clase `DiscoSimulado` que guarda bloques de tamaño fijo en un archivo binario y cuenta cada `leer_bloque(i)` y `escribir_bloque(i, datos)`. Está prohibido leer el archivo por fuera de esa clase. La memoria interna se limita a M registros (o M/B bloques en un búfer controlado por el equipo).

4.2 Escala de datos. Los datos se generan con un script propio y semilla fija. Tamaños orientativos:

- Componentes en memoria (C1 a C4): n entre 10^3 y 10^6 según la complejidad (para algoritmos exponenciales, $n \leq 20$).
- Memoria secundaria (C5): N entre 10^5 y 10^6 registros con $M \leq N/20$, de modo que los datos no quepan en memoria.
- Al menos 5 tamaños por experimento, en progresión geométrica (por ejemplo, n se duplica).

4.3 Protocolo de medición.

1. Medir con `time.perf_counter()`; repetir cada punto al menos 5 veces y reportar mediana y dispersión.
2. Separar la generación de datos del tiempo medido.
3. Graficar en escala log-log y estimar la pendiente; comparar con la predicha (por ejemplo, ≈1 para lineal, ≈2 para cuadrática).
4. Para n log n, graficar T(n)/(n log n) y verificar que tiende a una constante.
5. En C5, reportar E/S contadas frente a E/S predichas por la fórmula, no solo tiempo.

4.4 Repositorio. Estructura mínima: `README.md` con instrucciones de ejecución, una carpeta por componente, `datos/` con el generador, `experimentos/` con los scripts de benchmarking y pruebas unitarias básicas (pytest) para cada componente.

5. Cronograma y entregables

El avance de la sesión 2 cubre C1, C2 y C3; la entrega final de la sesión 4 cubre los seis componentes. Entre sesiones se espera trabajo autónomo (aprox. 5 h por estudiante por semana).

Sesión	Trabajo en laboratorio (2 h)	Trabajo autónomo antes de la siguiente sesión	Entregable	Peso en el proyecto
1	Lectura del caso, reparto de componentes, recurrencias y cotas preliminares, prueba de la clase DiscoSimulado	Implementar C1 y C2; generador de datos	Documento de diseño (2 a 3 páginas): componentes, estructuras elegidas y complejidades esperadas	Formativo
2	Revisión del avance con el docente; inicio de C4 y C5	Terminar C3, C4 y C5	**Avance:** código de C1, C2 y C3 con pruebas; análisis teórico de C1 a C3; plan de experimentos con hipótesis	Promediada con demás actividades del corte.
3	Experimentos de escala, conteo de E/S en C5, inicio de C6	Terminar C6, gráficas e informe	Repositorio preliminar y borrador del informe (revisión formativa)	Formativo
4	Sustentación: 10 a 12 min por equipo + 5 min de preguntas individuales	—	**Entrega final:** repositorio, informe técnico y diapositivas	Informe y código 10 %, sustentación 10 %

La entrega final se sube al aula virtual el día de la sesión 4 a más tardar a las 23:59 horas en el link de tarea de Teams indicado por su Docente. Las diapositivas no deben superar 12 láminas.

6. Casos por grupo

Cada caso concreta los seis componentes; el algoritmo nombrado es obligatorio, pero el equipo decide los detalles de implementación.

Grupo 1: Buscador de expedientes judiciales con recuperación de pasajes

Un índice invertido sobre un corpus sintético de expedientes permite buscar términos, corregir consultas mal escritas y seleccionar pasajes que caben en la ventana de contexto de un modelo de lenguaje.

Comp.	Algoritmo exigido	Uso en el caso
	mínima	Karp en instancias pequeñas

Bonificación: hashing lineal en disco para los estados de vehículos.

Grupo 3: Alineamiento de lecturas genómicas

Se buscan coincidencias aproximadas de lecturas cortas sintéticas de ADN sobre un genoma de referencia sintético usando k-mers como semillas.

Comp.	Algoritmo exigido	Uso en el caso
C1	Merge sort de k-mers + cota inferior $\Omega(n \log n)$ para ordenamiento por comparaciones	Construir el índice de semillas; resolver la recurrencia y explicar la cota con el árbol de decisión
C2	Arreglo dinámico de posiciones de coincidencia	Costo amortizado por el método contable
C3	Programación dinámica: Smith-Waterman (alineamiento local)	Alinear cada lectura contra la región candidata; $\Theta(mn)$ en tiempo
C4	Quicksort aleatorizado	Ordenar k-mers; análisis del número esperado de comparaciones y comparación empírica con C1
C5	Ordenamiento externo (external merge sort) de k-mers	Índice de un genoma que no cabe en M; contar E/S frente a la fórmula de ordenamiento externo
C6	Paralelismo tipo MapReduce con `multiprocessing`	Repartir lecturas entre p procesos (map) y combinar resultados (reduce); análisis de trabajo, profundidad, speedup y ley de Amdahl

Bonificación: árbol wavelet sobre $\Sigma = \{A, C, G, T\}$ con rank sobre vectores de bits.

Grupo 4: Detección de fraude en transacciones

Se evalúan transacciones sintéticas para marcar cuentas en lista negra y periodos de comportamiento anómalo, con presupuesto de latencia por transacción.

Comp.	Algoritmo exigido	Uso en el caso
C1	Subarreglo de suma máxima por divide y vencerás	Detectar el periodo más anómalo en la serie de desviaciones de una cuenta; comparar con Kadane $\Theta(n)$
C2	Tabla hash con duplicación de tamaño	Perfil por cuenta; costo amortizado de inserción
C3	Programación dinámica: mochila 0/1	Elegir el subconjunto de reglas de riesgo que maximiza la detección sin superar el presupuesto de latencia; mostrar que el voraz por razón no es

7. Contenido del informe técnico

El informe es un documento técnico libre (PDF o cuaderno Jupyter exportado en PDF), de máximo 15 páginas sin contar anexos, con estas secciones:

1. **Descripción del caso** (máx. 1 página): problema, supuestos y generador de datos.
2. **Arquitectura**: diagrama de los seis componentes y cómo se conectan.
3. **Análisis teórico por componente**: pseudocódigo breve, cotas O/Ω/Θ en mejor, peor y caso esperado; recurrencia y caso del Teorema Maestro (C1); método contable (C2); prueba de optimalidad o recurrencia de DP (C3); análisis probabilístico o de poda (C4); fórmula de E/S (C5); garantía formal (C6).
4. **Validación empírica**: para cada componente, la hipótesis, la gráfica (log-log cuando aplique), la pendiente o razón medida y la comparación con la teoría. En C5, tabla de E/S predichas frente a contadas.
5. **Discusión**: dónde coinciden teoría y práctica, dónde no y por qué (constantes, caché, intérprete de Python).
6. **Conclusiones y reparto del trabajo**: qué hizo cada integrante.
7. **Referencias**: en cualquier estilo consistente.

No se exige plantilla IEEE, APA ni ACM. Se valora la claridad, la corrección matemática y que cada afirmación de complejidad esté respaldada por una prueba o por un experimento.

8. Evaluación

La nota del proyecto (escala 0,0 a 5,0) es avance + entrega final + sustentación, y equivale a la nota del Corte 3.

8.1 Avance (sesión 2).

Criterio	Peso
C1: recurrencia correcta, caso del Teorema Maestro y código que funciona	35 %
C2: método contable correcto e implementación	25 %
C3: recurrencia de DP o prueba voraz e implementación	25 %
Plan de experimentos con hipótesis explícitas y generador de datos	15 %

8.2 Entrega final (sesión 4).

Criterio (U.C.)	Peso	Excelente (4,5 a 5,0)	Bueno (3,5 a 4,4)	Insuficiente (< 3,0)
Análisis teórico (2.2)	35 %	Cotas O, Ω y Θ correctas en los seis componentes, recurrencias resueltas, pruebas completas de optimalidad, amortización y garantía de C6	Análisis correcto con pruebas incompletas o sin caso esperado	Notaciones confundidas, recurrencias mal planteadas o pruebas

Criterio (U.C.)	Peso	Excelente (4,5 a 5,0)	Bueno (3,5 a 4,4)	Insuficiente (< 3,0)
				ausentes
Implementación (4.3)	35 %	Los seis componentes funcionan, código modular, con pruebas y README reproducible	Componentes funcionan con ineficiencias o poca modularidad	Faltan componentes o el código no ejecuta
Validación empírica y E/S (3.3)	30 %	Gráficas claras, pendientes o razones contrastadas con la teoría, E/S contadas frente a predichas, discrepancias explicadas	Experimentos presentes pero sin contraste con la teoría o sin conteo de E/S	Sin experimentos o datos sin metodología

8.3 Sustentación (sesión 4). 50 % presentación grupal (claridad, manejo del tiempo, diapositivas) y 50 % preguntas individuales. Cada estudiante responde preguntas sobre cualquiera de los seis componentes, no solo los que implementó. La nota individual de preguntas afecta la nota global de sustentación.

La franja entre 3,0 y 3,4 corresponde a desempeño aceptable con errores menores en algún criterio.

9. Integridad académica y uso de herramientas

Se permite usar bibliotecas estándar y NumPy, pero los algoritmos de C1 a C6 deben implementarse por el equipo; no se aceptan llamadas a `sorted`, `heapq`, `networkx`, `scikit-learn` u otras bibliotecas para resolver el núcleo de un componente (sí para generar datos o como referencia de comparación).

El uso de asistentes de IA para consultar o depurar debe declararse en el README. Las preguntas individuales de la sustentación verifican que cada integrante entiende el código y las pruebas entregadas; no poder explicarlos afecta la nota.